Cambridge (CIE) A Level Computer Science

Your notes

File Processing & Exception Handling

Contents

- File Handling
- Exception Handling

© 2025 Save My Exams, Ltd.

Your notes

File operations

What are file-processing operations?

- File-processing involves **interacting with external files** stored on secondary storage (e.g. hard drives, SSDs)
- In programming, files must be properly **opened**, **read/written**, and **closed** to ensure data is handled correctly
- Different **types of file access** are also used

Common file operations

Operation	Description
Open	The file is opened in a specific **mode**: • read – to read data • write – to overwrite data • append – to add new data to the end
Read	Data (e.g. lines or records) is read from the file into memory
Write	Data is written to the file (overwriting or appending depending on mode)
Close	Ends access to the file and ensures all data is saved properly

Types of file access

Access type	Description
Serial access	Records are **stored one after another**, and accessed in the **order they were added**. Often used for logging
Sequential access	Records are **read from the beginning to the end**, in order. Common for batch processing and text files
Random access	Records can be **accessed directly** using a **key or file position**. Common in databases or indexed files

Programming file-operations

Pseudocode

© 2025 Save My Exams, Ltd.

Basic file operations

```
OPEN filename FOR mode      // mode: READ, WRITE, APPEND
READ record FROM filename   // Read one record
WRITE record TO filename    // Write one record
CLOSE filename              // Always close the file
```

Serial access (e.g. logging events)

- Data is written **one after another** (no specific order)
- Often used with **APPEND** mode

Example: Writing logs serially

```
TYPE LogRecord
   DECLARE message : STRING
   DECLARE timestamp : STRING
ENDTYPE

DECLARE log : LogRecord

SET log.message = "System started"
SET log.timestamp = "12:00"

OPEN "logfile.txt" FOR APPEND
WRITE log TO "logfile.txt"
CLOSE "logfile.txt"
```

Sequential access (e.g. reading through all records)

- Records are **read in order** from start to end
- Common for reports or summaries

Example: Reading student grades sequentially

```
TYPE Student
   DECLARE name : STRING
   DECLARE grade : INTEGER
ENDTYPE

DECLARE student : Student

OPEN "students.txt" FOR READ

WHILE NOT EOF("students.txt")
   READ student FROM "students.txt"
   OUTPUT student.name, student.grade
ENDWHILE

CLOSE "students.txt"
```

Random access (e.g. accessing specific records by key)

- Accesses records **directly using a key** (e.g. record number or unique ID)
- Requires **indexed files** or fixed-length records

Example: Update a student's grade by record position

Your notes

```
TYPE Student
   DECLARE name : STRING
   DECLARE grade : INTEGER
ENDTYPE

DECLARE student : Student
DECLARE recordNumber : INTEGER

SET recordNumber = 5    // go directly to the 5th record

OPEN "students.txt" FOR RANDOM
SEEK "students.txt", recordNumber
READ student FROM "students.txt"

SET student.grade = 90  // update grade

SEEK "students.txt", recordNumber
WRITE student TO "students.txt"
CLOSE "students.txt"
```

Summary

Operation	Purpose	Example
OPEN	Opens a file in a specified mode	OPEN "file.txt" FOR READ
READ	Reads one record	READ student FROM "file.txt"
WRITE	Writes one record	WRITE student TO "file.txt"
CLOSE	Closes the file	CLOSE "file.txt"
SEEK	Moves to a specific record (random access)	SEEK "file.txt", 5

Python

Basic file operations

```
file = open("filename.txt", "r" or "w" or "a")  # Open for read, write, or append
line = file.readline()                          # Read one line
file.write(data)                                # Write one line
file.close()                                    # Close the file
```

Serial access (e.g. logging events)

Example: Appending log entries to a file (serial write)

```
from datetime import datetime

message = "System started"
timestamp = datetime.now().strftime("%H:%M")

log = f"{timestamp} - {message}\n"
```

Your notes

```
with open("logfile.txt", "a") as file:
    file.write(log)
```

Your notes

Sequential access (e.g. reading through all records)

Example: Reading through each student line by line

```
with open("students.txt", "r") as file:
    for line in file:
        name, grade = line.strip().split(",")
        print(f"Name: {name}, Grade: {grade}")
```

Random access (e.g. accessing specific records by key)

Example: Update a student's grade by index (record number)

```
record_number = 4  # 5th record (0-indexed)

with open("students.txt", "r") as file:
    records = file.readlines()

name, grade = records[record_number].strip().split(",")
records[record_number] = f"{name},90\n"  # Update grade to 90

with open("students.txt", "w") as file:
    file.writelines(records)
```

Summary

Operation	Python Example
Open	open("file.txt", "r")
Read	file.readline() / for line in file:
Write	file.write("text")
Close	file.close() or use with open(...)
Random access	lines = file.readlines() + list indexing

Java

Basic file operations

```
BufferedReader reader = new BufferedReader(new FileReader("filename.txt"));
String line = reader.readLine();
reader.close();

BufferedWriter writer = new BufferedWriter(new FileWriter("filename.txt", true)); // true = append
writer.write("Some text");
writer.newLine();
writer.close();
```

Serial access (e.g. logging events)

Example: Appending log entries to a file (serial write)

```
import java.io.*;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;

public class SerialAccessExample {
   public static void main(String[] args) throws IOException {
      String message = "System started";
      String timestamp = LocalTime.now().format(DateTimeFormatter.ofPattern("HH:mm"));
      String log = timestamp + " - " + message;

      BufferedWriter writer = new BufferedWriter(new FileWriter("logfile.txt", true));
      writer.write(log);
      writer.newLine();
      writer.close();
   }
}
```

Sequential access (e.g. reading through all records)

Example: Reading through each student line by line

```
import java.io.*;

public class SequentialAccessExample {
   public static void main(String[] args) throws IOException {
      BufferedReader reader = new BufferedReader(new FileReader("students.txt"));
      String line;

      while ((line = reader.readLine()) != null) {
         String[] parts = line.split(",");
         String name = parts[0];
         String grade = parts[1];
         System.out.println("Name: " + name + ", Grade: " + grade);
      }

      reader.close();
   }
}
```

Random access (e.g. accessing specific records by key)

Example: Update a student's grade by index (record number)

```
import java.io.*;
import java.util.*;

public class RandomAccessExample {
   public static void main(String[] args) throws IOException {
      List<String> records = new ArrayList<>();
      int recordNumber = 4; // 5th line (0-indexed)

      BufferedReader reader = new BufferedReader(new FileReader("students.txt"));
      String line;
      while ((line = reader.readLine()) != null) {
         records.add(line);
```

Your notes

```
        }
        reader.close();

        // Update the grade
        String[] parts = records.get(recordNumber).split(",");
        String updatedRecord = parts[0] + ",90";
        records.set(recordNumber, updatedRecord);

        // Overwrite the file
        BufferedWriter writer = new BufferedWriter(new FileWriter("students.txt"));
        for (String record : records) {
            writer.write(record);
            writer.newLine();
        }
        writer.close();
    }
}
```

Your notes

Summary

Operation	Java Example
Open (read)	BufferedReader reader = new BufferedReader(...)
Read	reader.readLine() or while ((line = ...) != null)
Write	BufferedWriter writer = new BufferedWriter(...)
Append	new FileWriter("file.txt", true)
Close	reader.close() / writer.close()
Random access	RandomAccessFile (complex) or simulate with List

Your notes

Exception handling

What is exception handling?

- An exception is an **unexpected event** that disrupts normal program execution
- Exception handling is **how a program detects and responds to these errors**, allowing it to **recover** or **shut down cleanly**

Why use exception handling?

- Prevents programs from **crashing unexpectedly**
- Allows for **user-friendly error messages**
- Makes programs more **robust** and **reliable**
- Handles problems caused by:
  - Division by zero
  - File not found or end-of-file errors
  - Hardware or device failures (e.g. printers, lost connections)
  - Invalid input from users

Causes of exceptions

Cause	Example
Programming errors	Uninitialised variables, logic errors
User errors	Entering text instead of a number
Hardware failure	Lost connection to printer, disk error

Pseudocode

```
TRY
   // Code that might cause an error
   OPEN "data.txt" FOR READ
   READ record FROM "data.txt"
   SET result = 10 / userInput
EXCEPT
   // Code to handle the error
   OUTPUT "An error occurred during file processing or division."
   CLOSE "data.txt"
ENDTRY
```

Key terms

© 2025 Save My Exams, Ltd.

Keyword	Purpose
TRY	Starts a block of code that may raise an error
EXCEPT	Executes if an error occurs in the TRY block
ENDTRY	Marks the end of the exception handling structure

Your notes

Python

```
try:
    file = open("data.txt", "r")
    line = file.readline()
    user_input = int(input("Enter a number: "))
    result = 10 / user_input
    print("Result:", result)
    file.close()

except FileNotFoundError:
    print("Error: File not found.")

except ZeroDivisionError:
    print("Error: Cannot divide by zero.")

except Exception as e:
    print("An unexpected error occurred:", e)

finally:
    try:
        file.close()
    except:
        pass  # file might not have opened
```

Java

```
import java.io.*;
import java.util.Scanner;

public class ExceptionHandlingExample {
    public static void main(String[] args) {
        BufferedReader reader = null;

        try {
            reader = new BufferedReader(new FileReader("data.txt"));
            String line = reader.readLine();

            Scanner scanner = new Scanner(System.in);
            System.out.print("Enter a number: ");
            int userInput = scanner.nextInt();

            int result = 10 / userInput;
            System.out.println("Result: " + result);

        } catch (FileNotFoundException e) {
            System.out.println("Error: File not found.");
```

```
    } catch (ArithmeticException e) {
        System.out.println("Error: Cannot divide by zero.");
    } catch (IOException e) {
        System.out.println("Error reading from file.");
    } catch (Exception e) {
        System.out.println("An unexpected error occurred: " + e.getMessage());
    } finally {
        try {
            if (reader != null) {
                reader.close();
            }
        } catch (IOException e) {
            // Ignore closing error
        }
    }
  }
}
```

Your notes